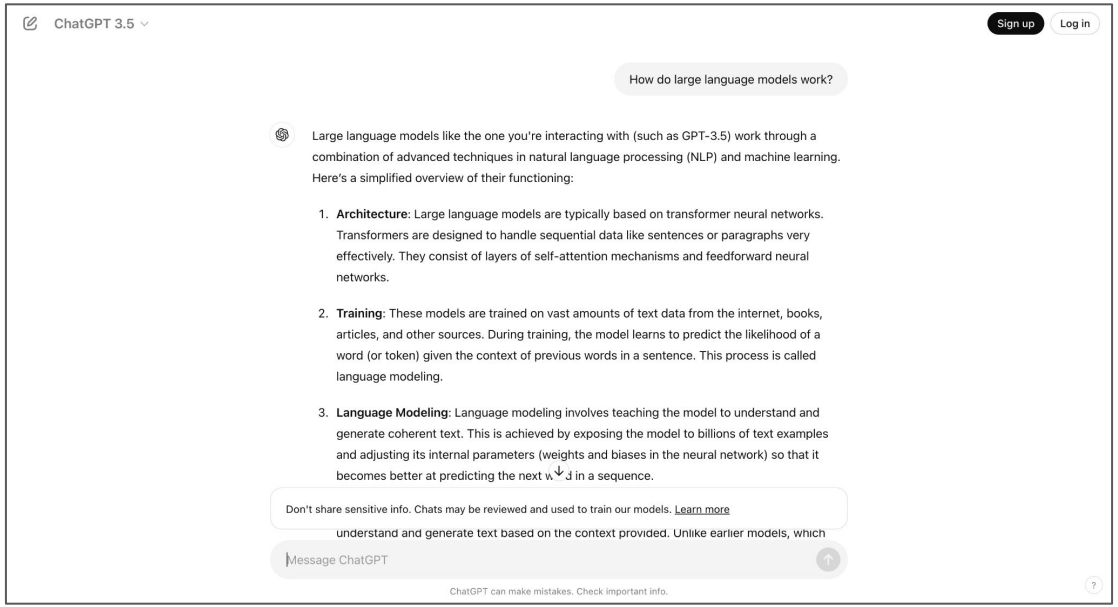
EECS 6694: GenAI and Modern Deep Learning

Instructor: Micah
Syllabus online, TA TBD

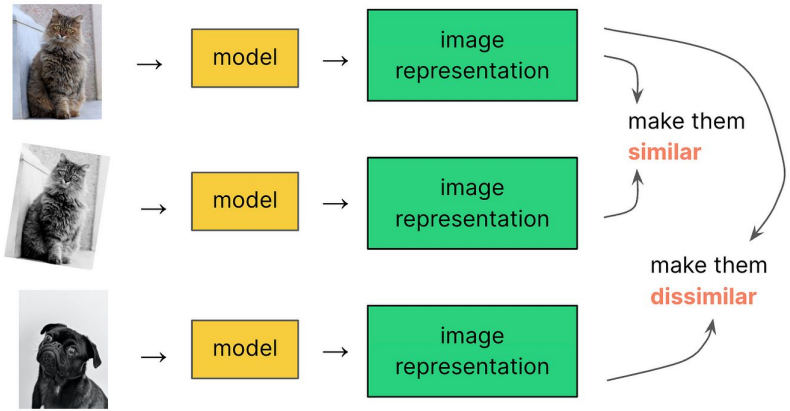
Contents of This Course

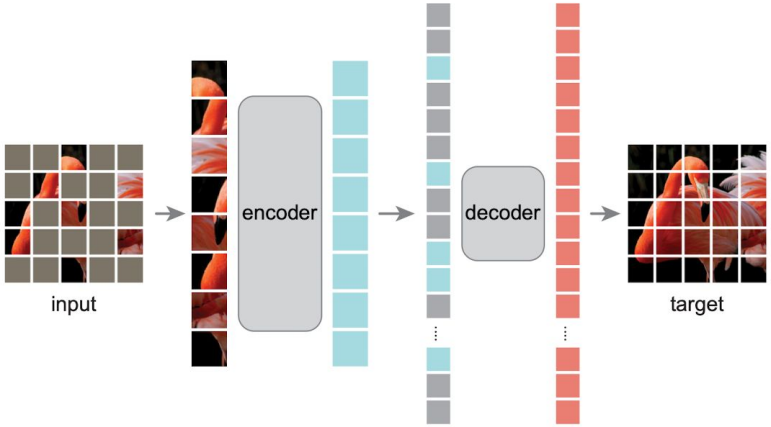
How to build and use language models



Contents of This Course

Self-supervised learning in vision



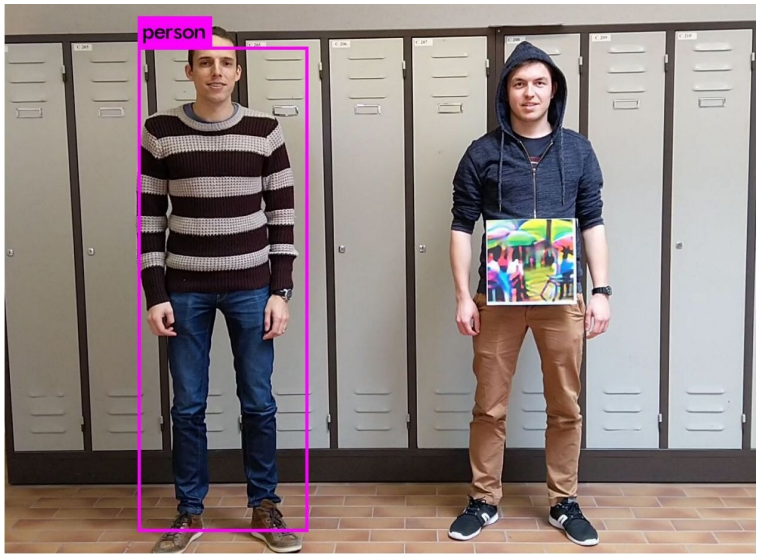


Contents of This Course

Generative models for images

Contents of This Course

Safety - security/privacy/fairness



Goals for the Semester

- Get up to speed on deep learning
- Learn how to read deep learning papers
- How to use Google for independent learning
- Useful tools you can apply elsewhere

Prerequisites

Introduction to machine learning

- Linear models
- Gradient descent
- Neural networks
- Backpropagation
- Training vs. validation vs. testing
- Python, PyTorch

Course Structure - Lectures

- Introductory lectures for each topic
- Guest lectures
- Student paper presentations

Course Structure - Assignments

- 65% of grade - Research Project
- 35% of grade - Paper presentation
- Non-graded - Coding tutorials, paper reading

The Research Project

Goals:

- Do novel research
- Write code
- Produce a paper

Possible content:

- Understand how something works
- Invent a new method
- Build a dataset

Keep in mind topics that you could be interested in!

Groups of ~3

A Crash Course on Deep Learning Basics (rushed and very dense)

The Components of a Deep Learning Pipeline

- The data
- The model
- Training
- Evaluation

The Components of a Deep Learning Pipeline

- The data
- The model
- Training
- Evaluation

The Data

- Independent samples from the same distribution:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$$

- x's are inputs, y's are labels
- Inputs will be a bunch of numbers
- Arranged into a vector or more axes like images
- Split randomly into training and test sets.
- Train on the train set, test on the test set.

The Components of a Deep Learning Pipeline

- The data
- The model
- Training
- Evaluation

The Model

- Takes in numbers
- Outputs numbers (predictions)
- Exactly what it predicts is determined by parameters
- Wiggling parameters changes what the model predicts

The Model

Outputs:

- Classification - categorical labels
  - Output is vector with one entry per class (e.g. what breed is this dog?)
  - The higher the value, the more confident we are that the input is in that class.
- Regression - numerical labels
  - Output is a number or multiple
  - e.g. predict life expectancy or predict life expectancy and income

Multi-Layer Perceptrons (MLPs)

One-hidden-layer MLP:

$$h = \sigma(W^1 x + b^1) \qquad \hat{y} = W^2 h + b^2$$

hidden layer output

Parameters $\theta = (W^1, b^1, W^2, b^2)$

Activation function σ allows neural nets to be more expressive than linear models.

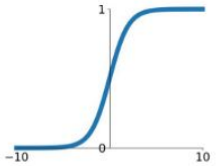
To add more layers, just alternate linear functions and nonlinearities:

$$\hat{y} = W^3 \sigma(W^2 \sigma(W^1 x + b^1) + b^2) b^3$$

Multi-Layer Perceptrons (MLPs) - Nonlinearities

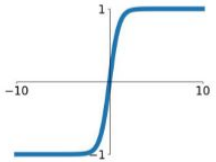
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



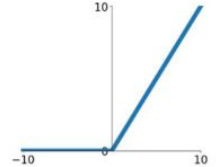
tanh

$$\tanh(x)$$



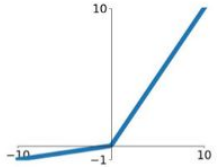
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

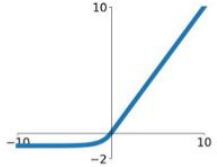


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Output of linear layer is a vector

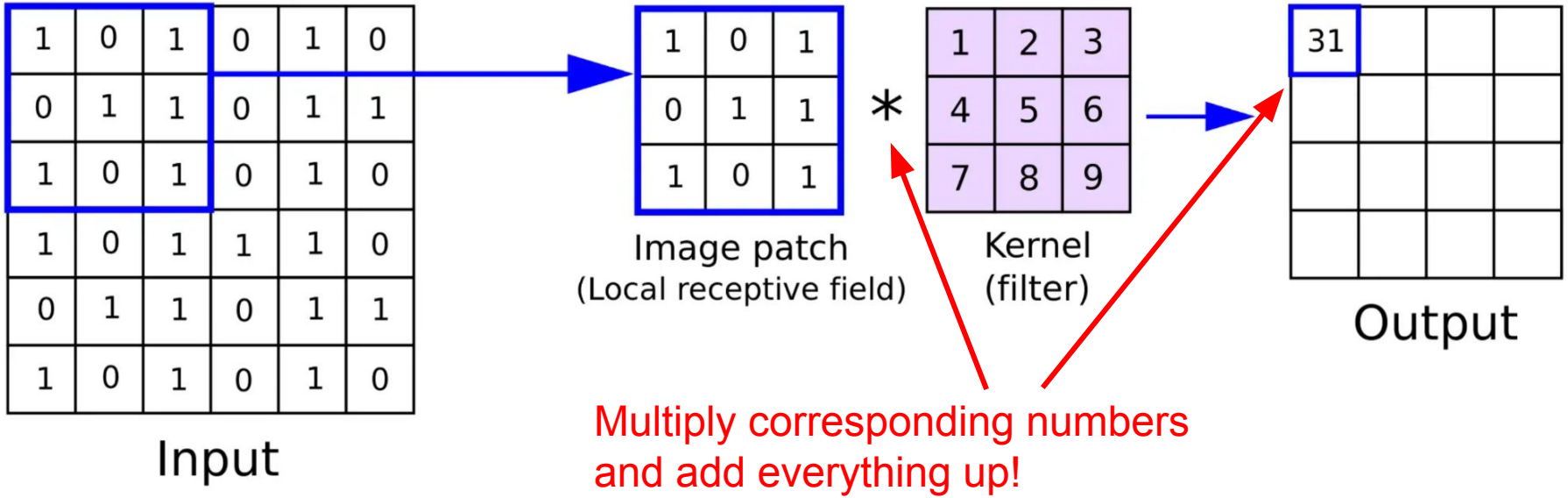
Apply activation function to vector entrywise

Convolutional Neural Networks (CNNs)

- Data that lies on grids
- Data with spatial dimensions - locality, translation equivariance
- Images
- Time series

Convolutional Neural Networks (CNNs)

A convolutional layer:

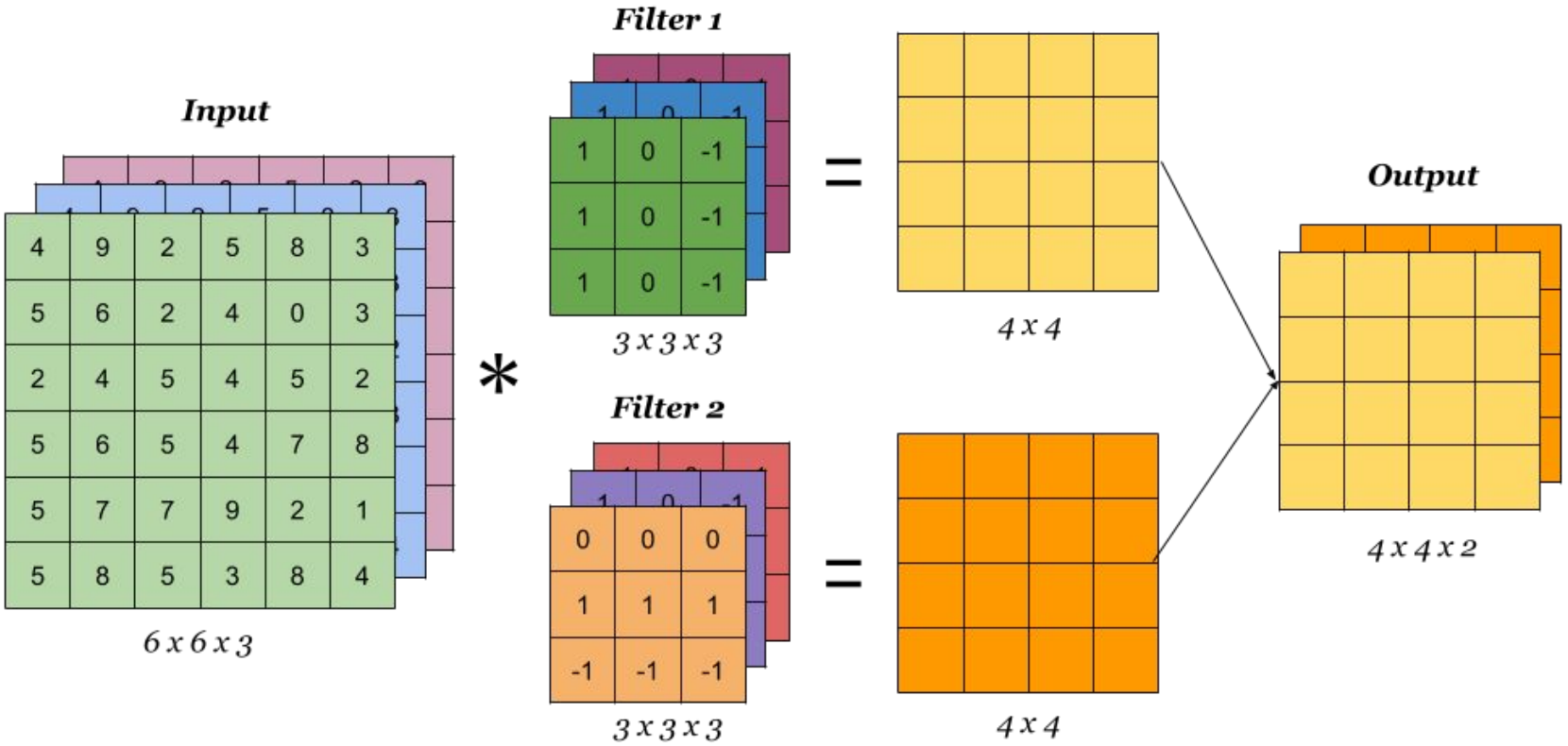


Convolutional Neural Networks (CNNs)

What if the input has multiple channels?
Solution: each convolutional filter has multiple channels too

How to output multiple channels?
Solution: use multiple filters

Convolutional Neural Networks (CNNs)

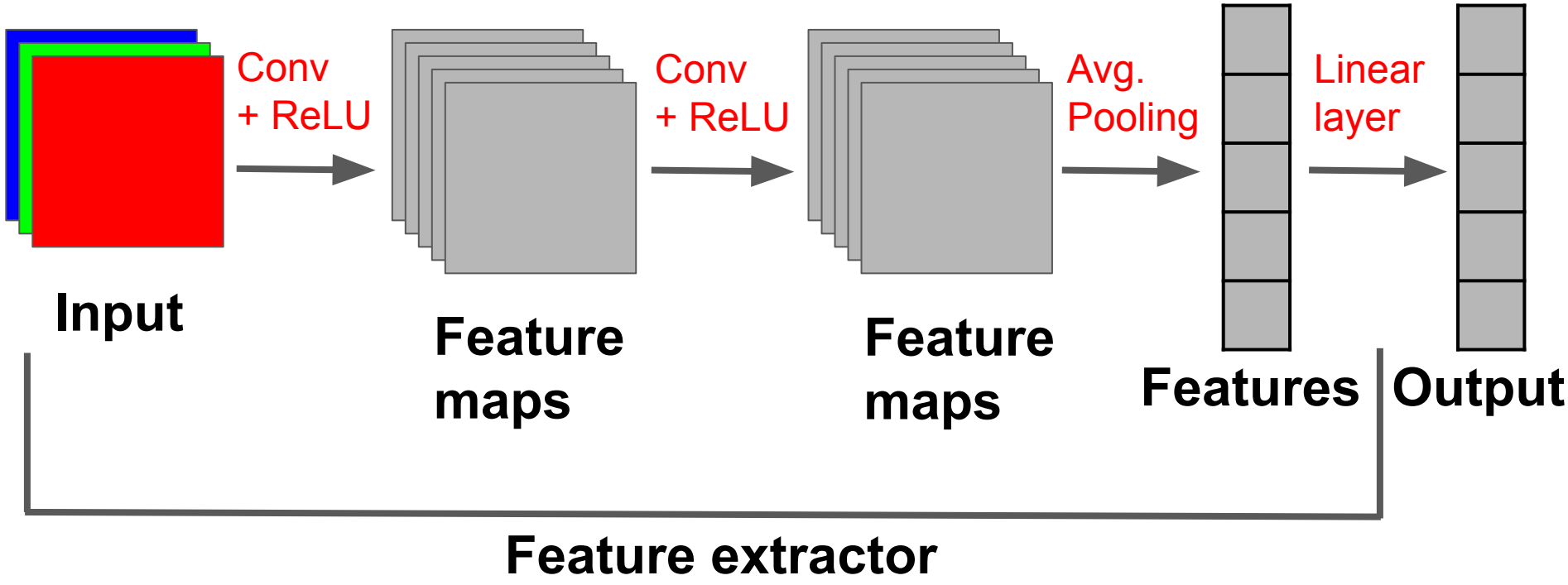


Convolutional Neural Networks (CNNs)

How do we make a basic convolutional neural network?

- Alternate convolutional layers and activation functions
- At end of stack, average entries of each channel (i.e. pooling)
- End up with vector of numbers. Call these **features**.
- Apply linear layer to features.

Convolutional Neural Networks (CNNs) - A Basic CNN



The Components of a Deep Learning Pipeline

- The data
- The model
- Training
- Evaluation

Training - The Loss Function

Training data $\mathcal{D}_{\text{train}} = \{(x, y)\}$

Step 1: loss function measures how close model's predictions are to the labels

Step 2: wiggle parameters so that the model's predictions are close to the labels

Low loss function means good predictions

Training - The Loss Function

Regression:

- Label is a number
- Model outputs a number
- Squared error:

$$(\hat{y} - y)^2$$

Training - The Loss Function

Classification:

- Label is the index of correct class
- Network outputs a vector with one entry per class, called **logits**
- Apply softmax to logits:

$$\text{softmax}(x) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Logits → probabilities. Higher logits → higher probabilities.
- Cross-entropy loss: measure how high probability (softmax) model assigns to the correct class.

$$\text{CE} = -\log(p_y)$$

Training - Setting up an Optimization Problem

Consider neural network f_θ

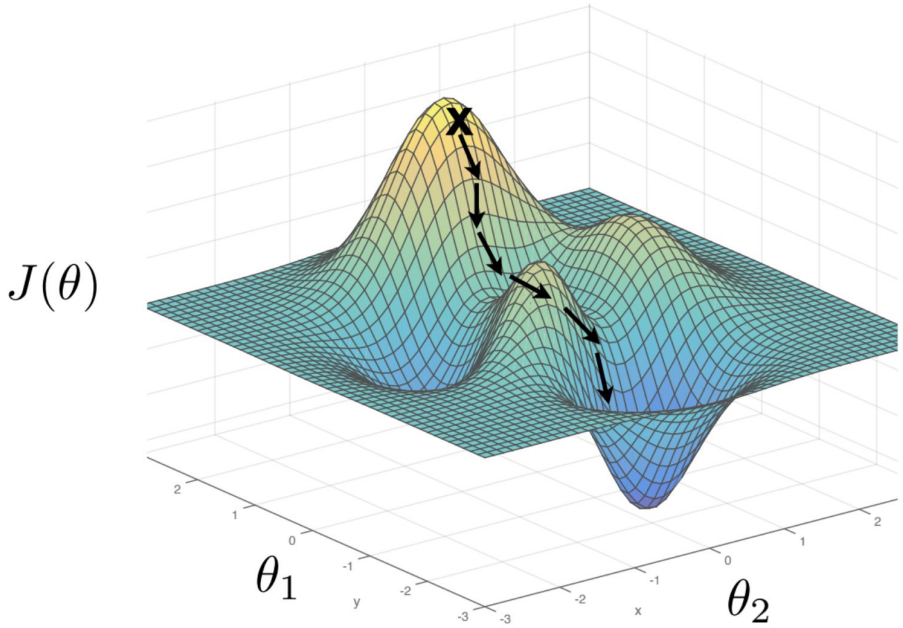
Compute loss for each sample in the training set $\mathcal{L}(f_\theta(x_i), y_i)$

Add them up $J(\theta) = \sum_{i=1}^{N} \mathcal{L}(f_\theta(x_i), y_i)$

When training loss is low, fit training set really well

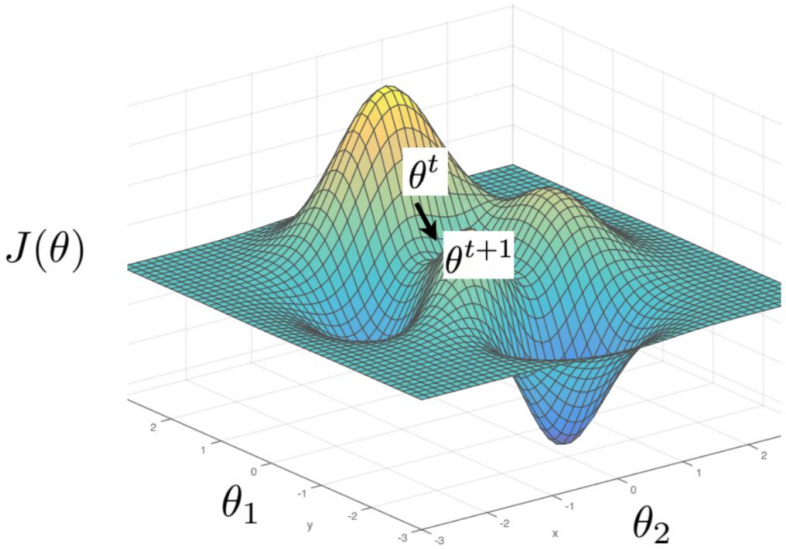
Find parameters that minimize training loss $J(\theta)$

Training - Gradient Descent



$$\theta^* = \arg\min_{\theta} J(\theta)$$

Training - Gradient Descent



$$\theta^* = \arg\min_{\theta} \underbrace{\sum_{i=1}^{N} \mathcal{L}(f_\theta(\mathbf{x}^{(i)}), \mathbf{y}^{(i)})}_{J(\theta)}$$

One iteration of gradient descent:

$$\theta^{t+1} = \theta^t - \eta_t \frac{\partial J(\theta)}{\partial \theta}\bigg|_{\theta=\theta^t}$$

learning rate

Training - Stochastic Gradient Descent (SGD)

- What if the training set is very very big?
- Can’t fit all data in memory at once
- Solution: train on a random mini-batch at a time
- Do gradient descent, compute loss on batch each step:

$$\sum_{i \in B} \mathcal{L}(f_\theta(x_i), y_i)$$

- Much much faster in practice whenever not enough memory for full batch

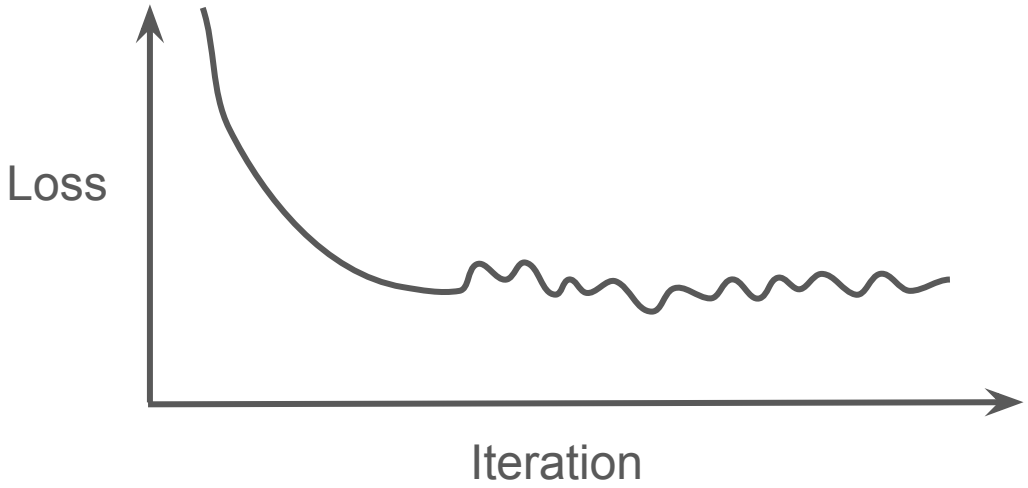
Training - Learning Rate Schedules

High learning rate - move fast, unstable

Low learning rate - move slow, stable

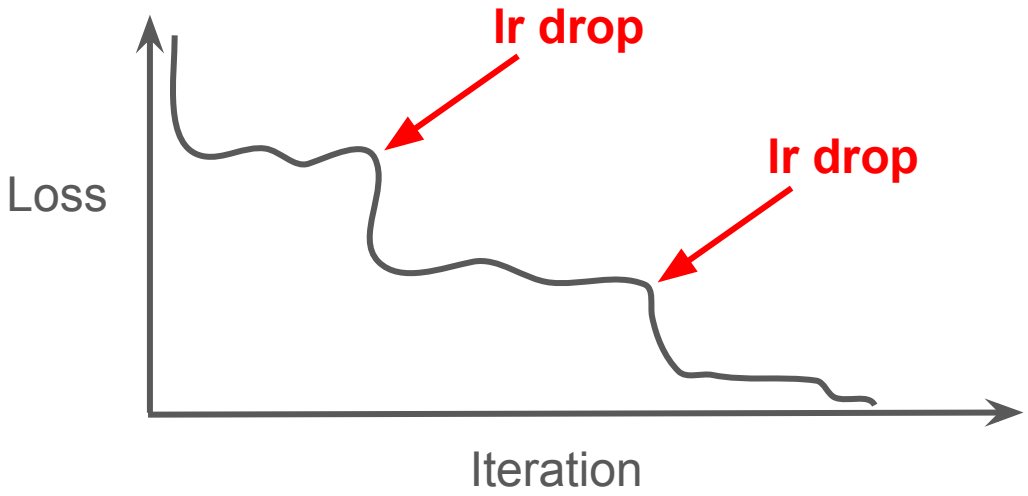
Choose highest learning rate that doesn't diverge

Problem: loss saturation



Training - Learning Rate Schedules

Solution: drop learning rate whenever loss saturates



New problem: manually tuning learning rate drops is a pain, expensive

Training - Automatic Learning Rate Schedules

Solution: decrease learning rate slowly over time

Benefits: robust, doesn't require much tuning

Example scheduler: cosine annealing

$$\frac{\mathrm{lr}_{\mathrm{init}}}{2}\left(1+\cos\left(\frac{\mathrm{iteration}_{\mathrm{curr}}}{\mathrm{iteration}_{\mathrm{last}}}\pi\right)\right)$$

The Components of a Deep Learning Pipeline

- The data
- The model
- Training
- Evaluation

Model Evaluation

Optimization vs. generalization

Train loss vs. test loss

Differentiable vs. non-differentiable performance metrics (e.g. loss vs. accuracy)

In-distribution vs. Out-of-Distribution (OOD)

Modern deep learning:

- Train huge model on tons of data
- Test on many downstream tasks

A Few Important Bells and Whistles

Normalization Layers

Internal covariate shift - outputs change, input range to next layer changes.

As you train, a layer becomes really good at using inputs in a certain range…

Then the input range changes…

Solution: normalize the output of every layer before feeding it in to the next one

Normalization Layers

Batch normalization:

- **Training**: After each convolutional layer:
  compute batch mean μ_i and standard deviation σ_i for each channel of output
- For each entry, compute $x \leftarrow \frac{x-\mu_i}{\sigma_i}$
- Multiply by learnable α_i and add β_i: $x \leftarrow \alpha_i x + \beta_i$
- **Inference**: How to handle inference (e.g. batch size 1)?
  keep moving average of μ_i and σ_i during training, use at inference.

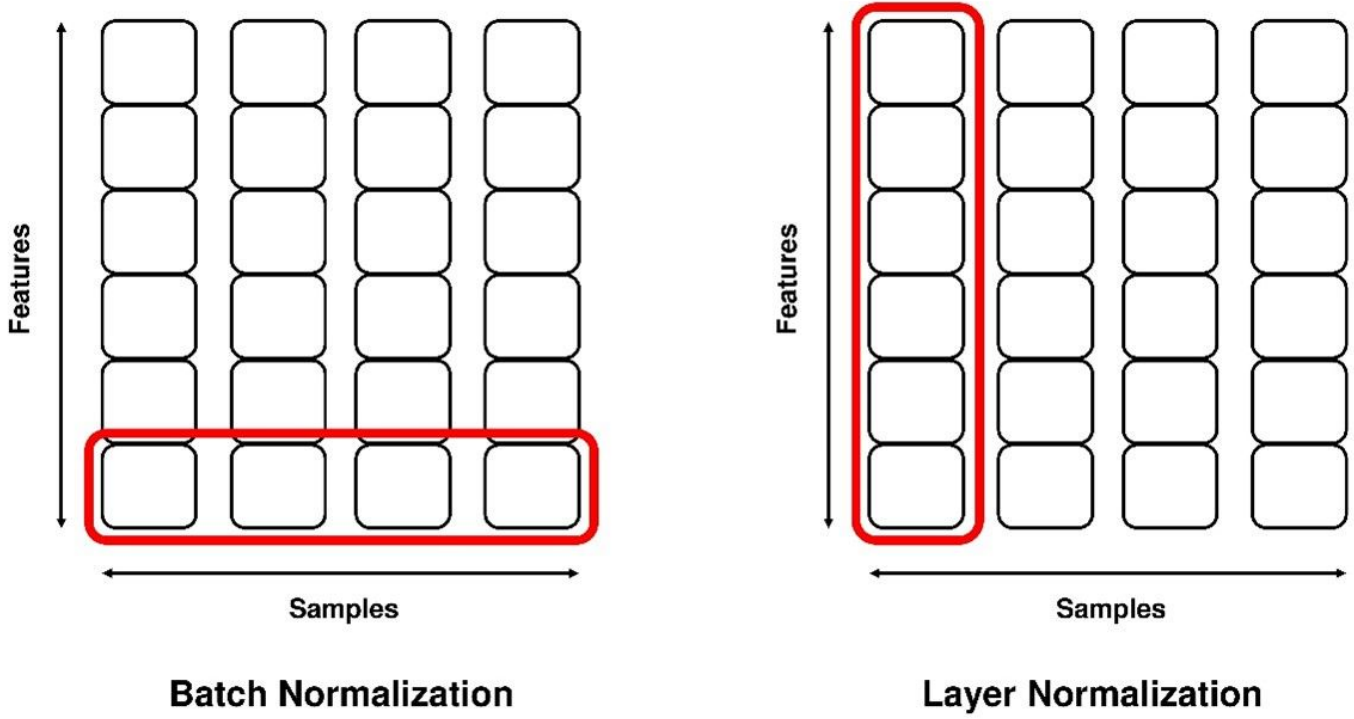
$$x \leftarrow \frac{x-\mu_i}{\sigma_i} \qquad x \leftarrow \alpha_i x + \beta_i$$

Normalization Layers

Layer normalization:

- Batch normalization computed batch statistics
- Normalize using individual sample, compute stats on entries instead of batch
- No more problems with inference time
- No moving averages needed
- Transformers

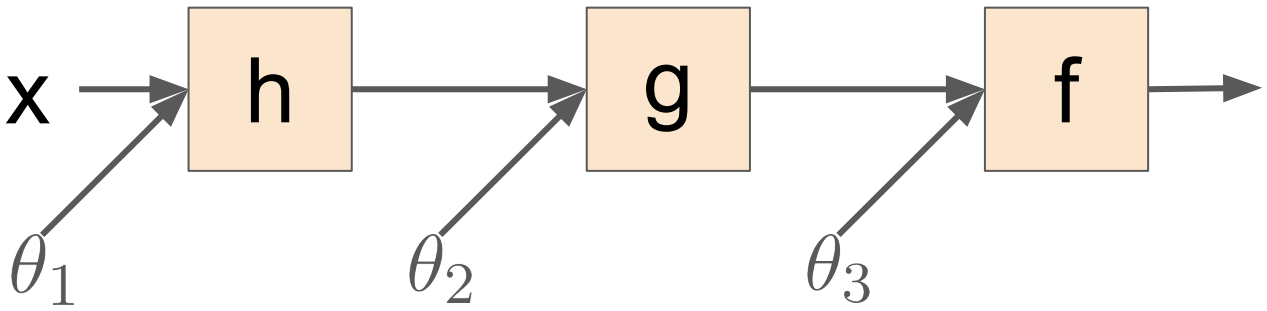
Normalization Layers



Vanishing Gradients

Consider computing gradients in a neural network.

$$\frac{d}{d\theta_1}[f(g(h(x,\theta_1),\theta_2),\theta_3)]$$
$$= f'(g(h(x,\theta_1),\theta_2),\theta_3)g'(h(x,\theta_1),\theta_2)h'(x,\theta_1)$$



Vanishing Gradients

Consider computing gradients in a neural network:

$$\frac{d}{d\theta_1}[f(g(h(x,\theta_1),\theta_2),\theta_3)]$$
$$= f'(g(h(x,\theta_1),\theta_2),\theta_3)g'(h(x,\theta_1),\theta_2)h'(x,\theta_1)$$

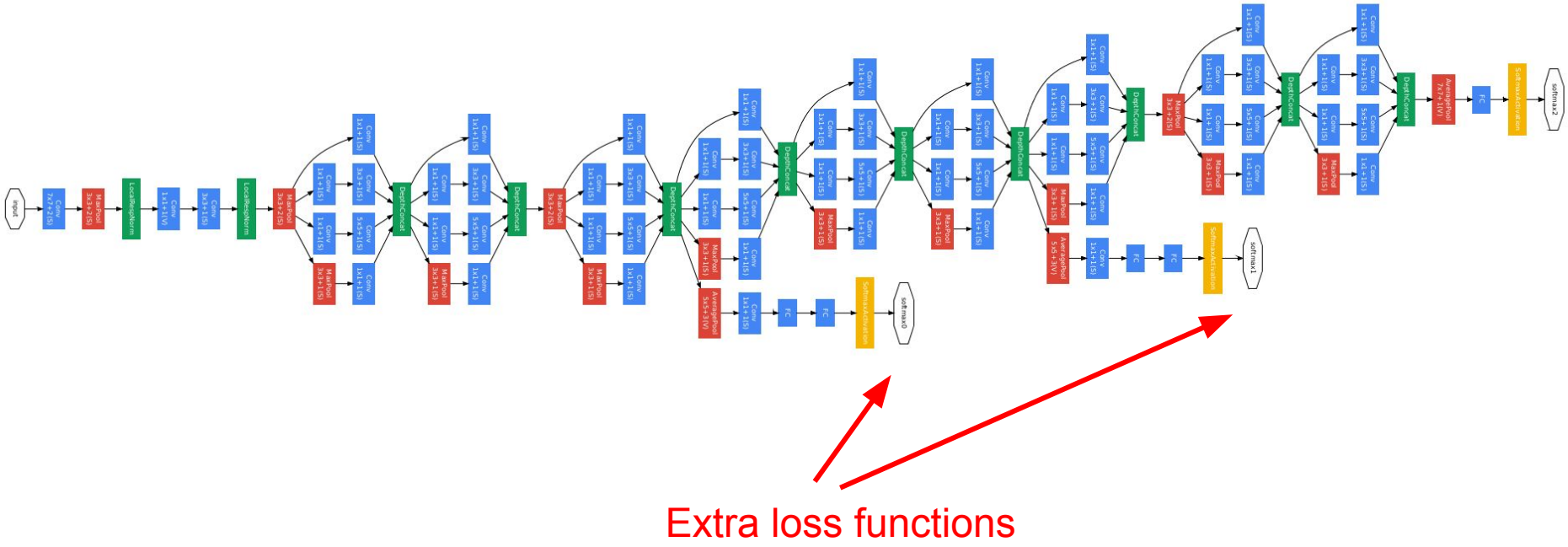
When computing gradients in a deep network, we multiply a lot of things together.

If they have magnitude less than 1, then the gradients will be tiny. Slow training.

If they are big, the gradients will be huge. Unstable training.

More depth means more vanishing.

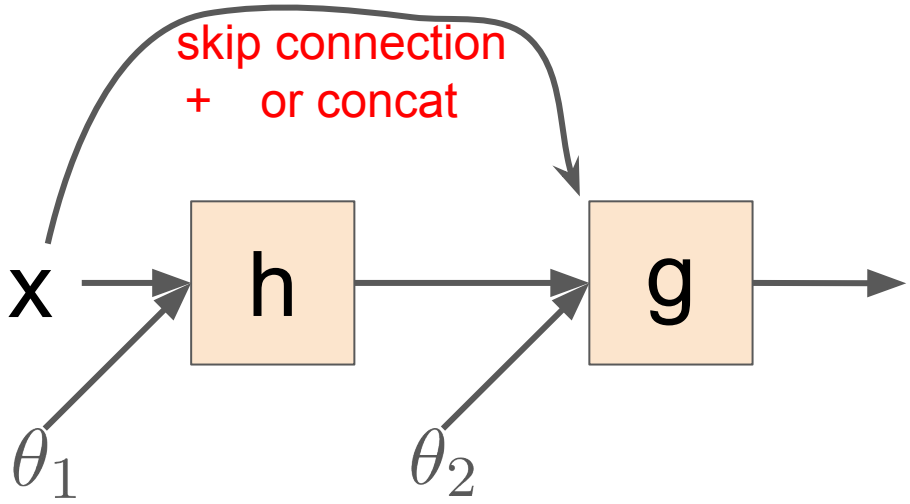
Vanishing Gradients - GoogLeNet



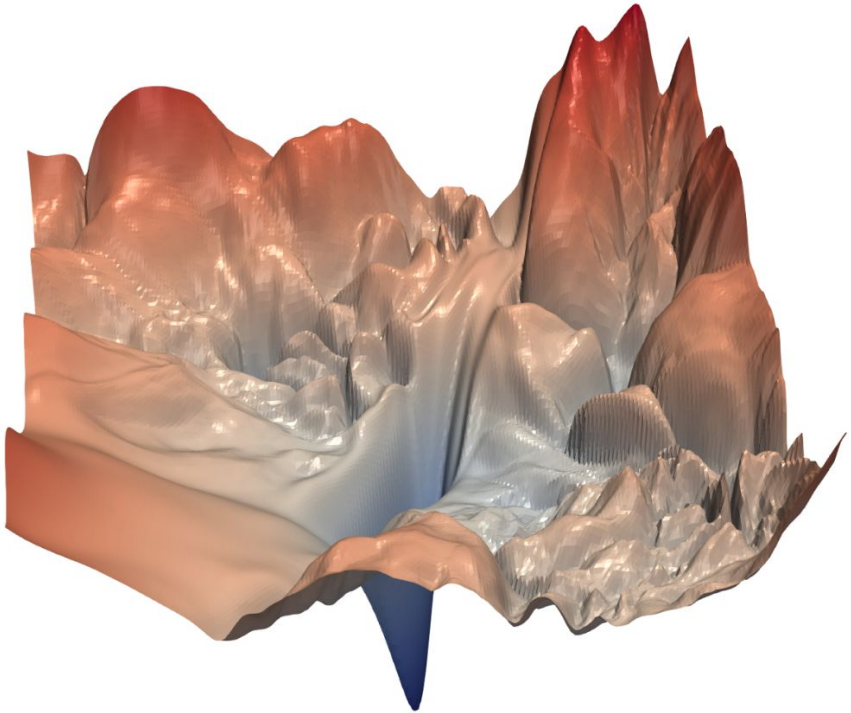
Skip Connections

Skip layers (addition vs. concatenation) - gradients no longer vanish

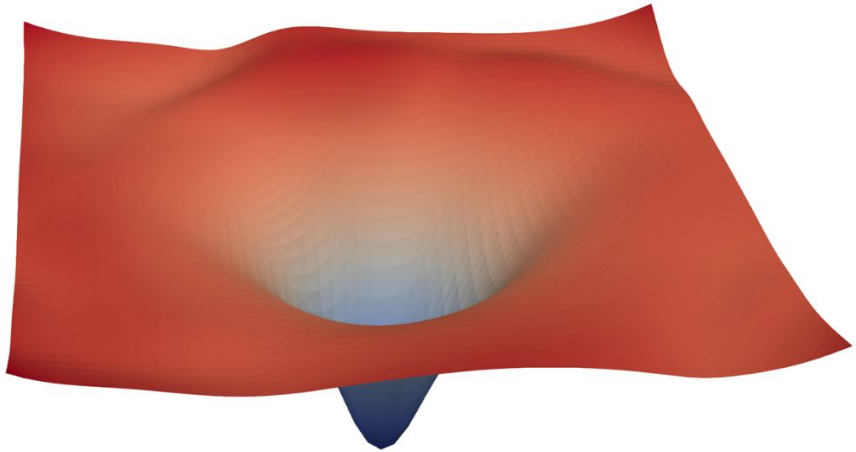
$$\frac{d}{dx}[g(h(x)+x)] = g'(h(x)+x)h'(x) + g'(h(x)+x)$$



Skip Connections

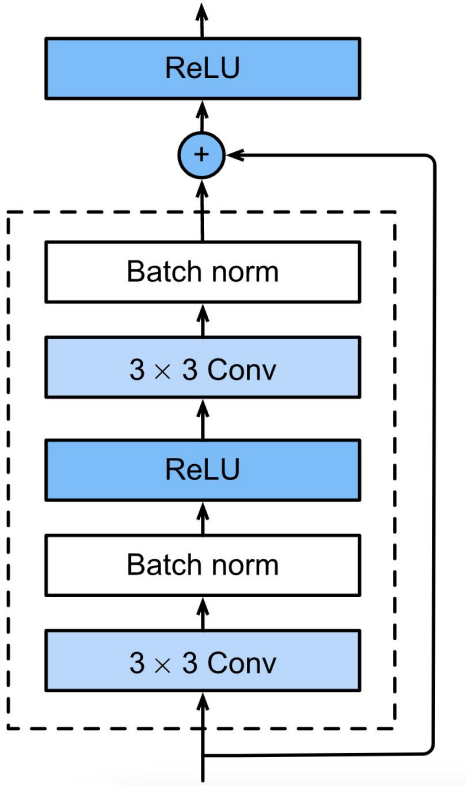


(a) without skip connections



(b) with skip connections

Putting It All Together - ResNets



Putting It All Together - ResNets

The Secret Sauce for Neural Network Design

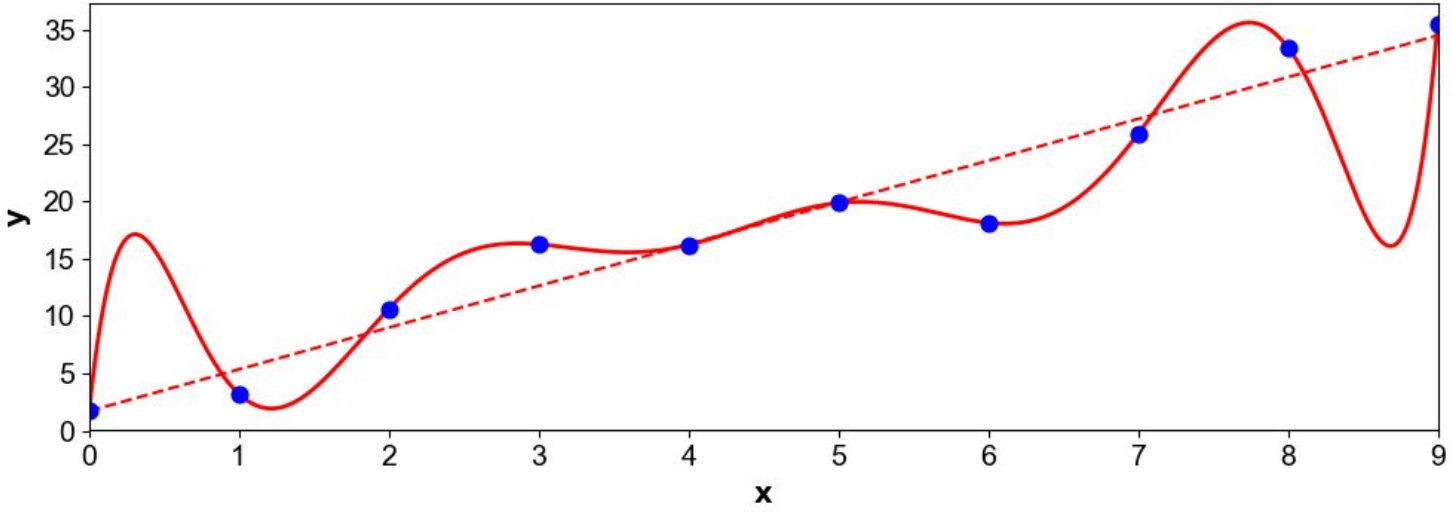
- Flexibility
- Easy to optimize
- Inductive bias

Inductive Bias

Lots of possible functions fit our training data. Most fail on data we haven't seen.

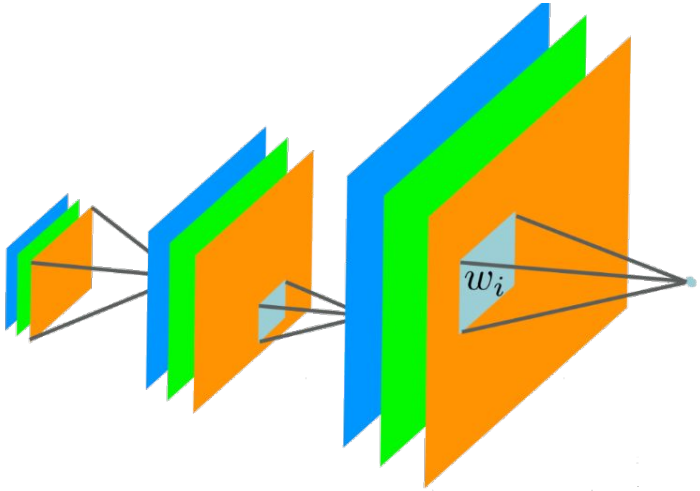
Inductive bias - the preference for choosing certain functions over others.

Good machine learning models extrapolate to new data, not just train data.



Deep Image Prior (Ulyanov 2017)

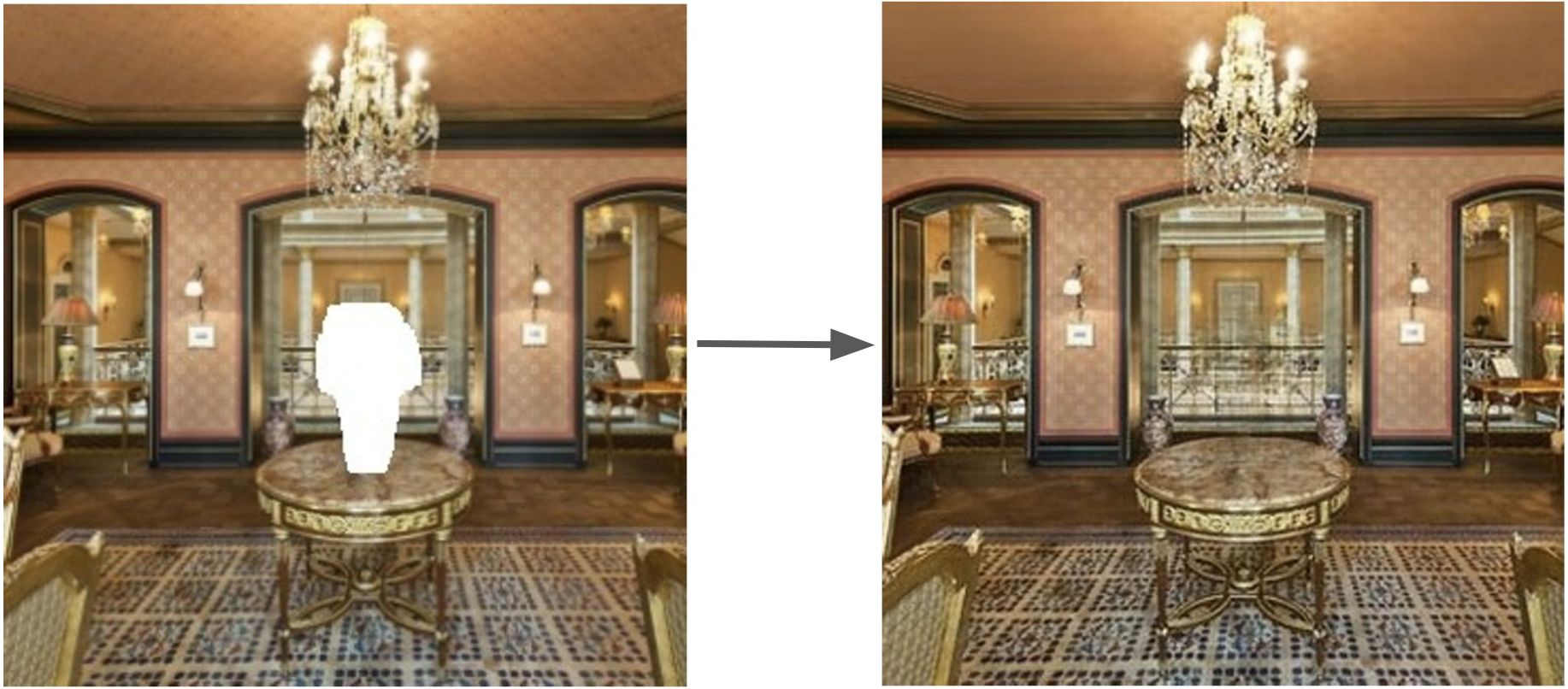
random vector (frozen)



Corrupted

$$\min_{\text{network params}} \ell(\text{network output}, \text{known pixels})$$

Deep Image Prior (Ulyanov 2017)



Deep Image Prior (Ulyanov 2017)